

Rapport complet du projet d'optimisation numérique Couplage FreeFEM++ / Python

Laurent ZHU – Alexandre LE DROUCPEET

19 mai 2026

Table des matières

1	Introduction	2
2	Description du code	2
2.1	Code Python (notebook optimize2.ipynb)	2
2.2	Code FreeFEM (heat_solver.edp)	3
3	Optimisation des performances	3
3.1	Problème initial : temps d'évaluation trop élevé	3
3.2	Réduction du maillage	3
3.3	Ajustement du budget d'évaluations	3
4	Restructuration du dépôt GitHub	3
5	Proposition d'interface utilisateur	4
6	Synthèse des travaux réalisés	5
7	État des lieux et perspectives	5
7.1	État actuel	5
7.2	Perspectives	5
8	Conclusion	5
A	Exemple de fichier .env	5
B	Extrait du code FreeFEM modifié (maillage)	6
C	Suite du projet	6
C.1	Influence du maillage sur l'optimum	6
C.2	Test de sensibilité au point initial	6
C.3	Détermination des hyperparamètres optimaux pour chaque méthode	6
C.4	Plan d'exécution	7

1 Introduction

Ce projet vise à développer une plateforme d'optimisation automatique pour un problème bidimensionnel de conduction thermique stationnaire. Le solveur éléments finis FreeFEM++ calcule la température sur un domaine avec ailettes, et la fonction objectif J est la température moyenne sur les surfaces des ailettes. L'objectif est de maximiser J (meilleure performance thermique) en faisant varier six paramètres : les conductivités thermiques $k_1, \dots, k_5$ de cinq couches de matériau et le nombre de Biot Bi .

La plateforme couplée fonctionne en boucle : Python génère un vecteur $\mathbf{x} = (k_1, \dots, Bi)$, le passe à FreeFEM via un fichier texte, FreeFEM résout l'EDP, écrit J dans un fichier, et Python lit cette valeur pour guider l'optimisation.

2 Description du code

2.1 Code Python (notebook optimize2.ipynb)

Le code est organisé en cellules :

1. **Importation** : `numpy`, `scipy.optimize`, `subprocess`, `tqdm`, `dotenv`, etc.
2. **Chemins** : lecture de `FREEFEM_PATH` depuis `.env`.
3. **Fonctions d'interface FreeFEM** :
 - `write_params(x)` : écrit les 6 valeurs dans `params.txt`.
 - `run_freefem(command, arg)` : exécute FreeFEM avec l'option `-doplot 0/1`.
 - `read_objective()` : lit `objective.txt` et retourne J .
4. **Historique et meilleur design** : variables globales `history`, `best_J`, `best_x`, fonction `save_best_design()`.
5. **Fonction d'évaluation** `evaluate(x)` :
 - Lancement du chronomètre.
 - Écriture des paramètres.
 - Appel FreeFEM (sans affichage).
 - Lecture de J .
 - Stockage dans `history`.
 - Mise à jour du meilleur design.
 - Retourne $-J$ pour minimisation.
6. **Sauvegarde et visualisation** : `save_history()`, `plot_convergence()`.
7. **Méthodes d'optimisation avec barres de progression** :
 - `run_differential_evolution(bounds, maxiter, popsize)`
 - `run_nelder_mead(bounds, maxiter)`
 - `run_basinhopping(bounds, niter)`Chacune réinitialise l'historique, utilise `tqdm` pour afficher la progression, mesure le temps, et retourne un dictionnaire de résultats (meilleur J , nombre d'évaluations, temps, succès).
8. **Programme principal** : évaluation du design initial $x_0 = (0.5, \dots, 0.5)$, définition des bornes, exécution séquentielle des trois méthodes, génération d'un tableau comparatif Pandas, sauvegarde en CSV et affichage de la convergence.

2.2 Code FreeFEM (heat_solver.edp)

Le script lit `params.txt`, construit le domaine (rectangle avec cinq couches verticales et ailettes), résout l'équation de la chaleur :

$$-\nabla \cdot (k \nabla T) = 0$$

avec conditions aux limites : flux imposé sur les ailettes, échange convectif (Bi) sur les faces latérales, et température imposée à la base. Il calcule J comme la température moyenne sur toutes les surfaces des ailettes et écrit cette valeur dans `objective.txt`.

3 Optimisation des performances

3.1 Problème initial : temps d'évaluation trop élevé

Lors des premiers tests, le maillage FreeFEM était de 200×200 éléments. Chaque évaluation prenait **45 secondes**. Avec un budget de 150 évaluations par méthode, l'optimisation complète dépassait deux heures, ce qui était rédhibitoire.

3.2 Réduction du maillage

En modifiant le script FreeFEM pour utiliser un maillage de 50×50 (instruction `mesh Th = square(50,50);`), le temps par évaluation est tombé à **2–3 secondes** (gain d'un facteur 15 à 20). La précision de J reste suffisante pour l'optimisation (les gradients et les optimums sont correctement représentés). Cette accélération rend l'optimisation interactive et permet de comparer plusieurs méthodes.

3.3 Ajustement du budget d'évaluations

Pour obtenir une réponse rapidement, nous recommandons :

- **Differential Evolution** : `maxiter=2, popsize=2` → 36 évaluations (environ 1,5 minute).
- **Nelder-Mead** : `maxiter=30` → environ 80 évaluations (3–4 minutes).
- **Basinhopping** : `niter=10` avec minimiseur local limité à 10 itérations → environ 100–150 évaluations (5–7 minutes).

Ces réglages permettent d'obtenir une solution satisfaisante en moins de 10 minutes.

4 Restructuration du dépôt GitHub

Pour une meilleure maintenabilité et réutilisabilité, le dépôt sera réorganisé selon l'arborescence suivante :

```
1 heat-cond-optim/  
2     README.md                                # description,  
   installation, utilisation  
3     requirements.txt                          # packages Python (scipy,  
   tqdm, pandas, matplotlib, python-dotenv)  
4     .env.example                             # exemple de fichier d'  
   environnement  
5     .gitignore
```

```

6      src/
7          __init__.py
8          main.py                # programme principal (
orchestration)
9          freefem_interface.py   # write_params,
run_freefem, read_objective
10         optimization.py        # evaluate, reset,
wrappers des methodes
11         visualization.py       # plot_convergence,
save_history
12         utils.py               # gestion des dossiers,
logging
13     scripts/
14         heat_solver.edp         # script FreeFEM
15     results/                    # stockage des sorties (
best_design.txt, *.csv, *.png)
16     config/
17         opt_config.json         # parametres par default
(bornes, maxiter, popsize...)
18     tests/
19         test_freefem.py         # tests unitaires (mocking
)

```

Listing 1 – Structure proposée

Les noms sont explicites : `freefem_interface.py` isole les appels système, `optimization.py` centralise la logique d’optimisation, `visualization.py` gère les graphes. Le fichier `.env.example` documente la variable `FREEFEM_PATH`.

5 Proposition d’interface utilisateur

Pour démocratiser l’outil, une interface graphique (Streamlit) offrira les fonctionnalités suivantes :

- **Panneau de contrôle :**
 - Sliders ou champs numériques pour $k_1, \dots, k_5, Bi$ (avec bornes).
 - Sélection du maillage : grossier (25×25), moyen (50×50), fin (100×100). Un temps estimé s’affiche.
 - Bouton “Évaluer” : lance FreeFEM une fois, affiche J et une figure de la température (option `-doplot 1`).
- **Optimisation :**
 - Choix de la méthode (DE, Nelder-Mead, Basinhopping).
 - Réglage du budget (maxiter, popsize).
 - Barre de progression en temps réel.
 - Courbe de convergence dynamique (meilleur J en fonction des évaluations).
- **Résultats :**
 - Affichage des meilleurs paramètres et de la valeur J .
 - Téléchargement de l’historique (CSV) et de la figure.

Un prototype avec `streamlit` sera développé, accessible via navigateur, sans installation Python pour l’utilisateur final (si déployé sur un serveur). L’interface communiquera avec le même backend que le notebook.

6 Synthèse des travaux réalisés

- Mise en place d'un couplage fonctionnel Python–FreeFEM++.
- Implémentation de trois méthodes d'optimisation (DE, Nelder-Mead, basinhopping) avec mesure des performances.
- Réduction drastique du temps d'évaluation ($45s \rightarrow 2-3s$) par affinement du maillage.
- Ajout de barres de progression avec `tqdm` pour un suivi convivial.
- Production d'un tableau comparatif (meilleur J , nombre d'évaluations, temps CPU, succès).
- Proposition d'une architecture modulaire et d'une interface utilisateur Streamlit.

7 État des lieux et perspectives

7.1 État actuel

La plateforme est opérationnelle pour des optimisations de petite à moyenne taille (quelques centaines d'évaluations). Le maillage moyen (50×50) donne un bon compromis précision/temps. Les résultats sont reproductibles et sauvegardés. Le code est prêt à être restructuré selon le plan défini.

7.2 Perspectives

1. **Optimisation multi-méthodes automatique** : exécuter en parallèle plusieurs algorithmes et sélectionner le meilleur résultat.
2. **Intégration du maillage variable** : passer le paramètre de résolution au script FreeFEM via un argument ou un fichier.
3. **Méta-optimisation** : régler automatiquement les hyperparamètres de DE (mutation, recombinaison) par une recherche sur grille réduite.
4. **Déploiement web** : mettre l'interface Streamlit en ligne pour une utilisation externe.
5. **Extension à d'autres problèmes physiques** : adapter le script FreeFEM pour l'élastostatic, l'écoulement potentiel, etc.

8 Conclusion

Ce projet démontre la faisabilité d'un couplage Python–FreeFEM++ pour l'optimisation de systèmes thermiques. Grâce à des optimisations de maillage et à des réglages adaptés, les temps de calcul deviennent acceptables pour une utilisation interactive. La refonte modulaire et l'interface utilisateur proposée transformeront cet outil de recherche en une application robuste et accessible.

A Exemple de fichier `.env`

```
1 FREEFEM_PATH=C:\Program Files\FreeFem++\FreeFem++.exe
```

B Extrait du code FreeFEM modifié (maillage)

```
1 mesh Th = square(50, 50); // anciennement 200x200
2 ... // reste du script inchang
```

C Suite du projet

Afin d’approfondir l’analyse et de valider la robustesse de la plateforme, plusieurs études complémentaires sont prévues.

C.1 Influence du maillage sur l’optimum

Un maillage grossier (50×50) accélère les calculs mais pourrait déplacer légèrement l’optimum ou introduire un biais. Pour quantifier cet effet, nous allons :

- Lancer une optimisation complète avec un maillage **fin** (200×200) pendant une nuit (environ 8 heures, soit 150 évaluations à 45 s chacune).
- Comparer le meilleur design obtenu avec celui du maillage moyen (50×50 , 2–3 s par évaluation).
- Si les deux optima sont proches (à moins de 5% près), l’utilisation du maillage grossier sera justifiée pour les explorations rapides. Sinon, il faudra trouver un compromis (ex. 100×100).

C.2 Test de sensibilité au point initial

Les méthodes basées sur un seul point de départ (Nelder-Mead, minimiseur local de basinhopping) peuvent converger vers des optima différents selon le point initial. Nous allons :

- Faire varier systématiquement le point de départ x_0 en tirant aléatoirement 10 points dans les bornes.
- Exécuter Nelder-Mead et basinhopping depuis chacun de ces points (avec maillage 50×50 , budget limité à 100 évaluations).
- Analyser la dispersion des résultats (moyenne, écart-type du meilleur J atteint).
- Déterminer si un point initial particulier (par exemple le design initial $k_i = 0.5$, $Bi = 0.5$) est suffisamment représentatif ou s’il faut systématiquement utiliser une stratégie globale (DE).

C.3 Détermination des hyperparamètres optimaux pour chaque méthode

Les performances de Differential Evolution dépendent fortement de ses paramètres : taille de la population (*popsiz*e), facteur de mutation (*mutat*ion), probabilité de croisement (*recomb*ination), et stratégie. Un réglage par défaut n’est pas forcément optimal pour notre fonction objectif bruitée.

Nous allons mener une ****optimisation des hyperparamètres**** (méta-optimisation) sur un budget limité (200 évaluations) :

- **DE** : faire varier $popsiz$ $\in [2, 10]$, $mutation \in [0.3, 1.2]$, $recombination \in [0.5, 0.9]$, stratégies (best1bin, rand1bin, best2bin, etc.). Critère : meilleur J atteint après 100 évaluations.
- **Nelder-Mead** : paramètres α (réflexion), γ (expansion), ρ (contraction), σ (réduction). Utiliser les valeurs par défaut de SciPy semble satisfaisant, mais on pourra tester la sensibilité.
- **Basinhopping** : nombre d'itérations, facteur de température (T), pas de la marche aléatoire (step size), et options du minimiseur local (maxiter, tolérance).

À l'issue de cette étude, nous fixerons des jeux de paramètres recommandés pour chaque méthode, garantissant le meilleur compromis entre vitesse de convergence et robustesse pour notre problème.

C.4 Plan d'exécution

1. **Nuit 1** : optimisation avec maillage fin (200×200) via DE (maxiter=20, popsize=6 $\rightarrow$ 420 évaluations, temps estimé 5h). Sauvegarde du meilleur design.
2. **Nuit 2** : tests de sensibilité aux points initiaux (10 exécutions de Nelder-Mead, 10 de basinhopping, maillage 50×50).
3. **Nuit 3** : méta-optimisation des hyperparamètres de DE (recherche sur grille ou bayésienne).
4. **Analyse finale** : comparaison des optimums, rédaction d'une recommandation de paramétrage.

Ces travaux permettront de livrer une plateforme non seulement fonctionnelle, mais aussi calibrée au mieux pour le problème HEAT-COND.